AIT Asian Institute of Technology

NLP - Project Proposal

RepoQA

A Hierarchical Retrieval-Augmented System for Repository-Level Code Question Answering

March 2026

Presented by **Sushmi Shrestha**
[ST126526]

Context & Problem

- **Situation:**
  Developers spend 58–70% of their time reading and understanding code, not writing it (Xia et al., 2018)

  Questions developers ask every day:

  "How does authentication work in this project?"
  "What does this middleware do and why?"
  "How is Docker configured for production?"
  "Why was this retry pattern chosen?"

- **The Problem:**
  Current AI tools (Copilot, Cursor) excel at code completion but cannot answer understanding questions

  Evidence: CoReQA benchmark (Chen et al., 2025): GPT-4o + BM25 = 6.91/10 accuracy, 6.34/10 completeness

  Root cause: BM25 keyword matching misses semantic relationships in code - adding BM25 only improved accuracy by 0.06 points

- **The Research Gap:** CoReQA identifies the retrieval problem but does not propose a solution.
  No existing work combines hierarchical understanding + hybrid retrieval + question-type routing for code Q&A.

Approach & Contribution

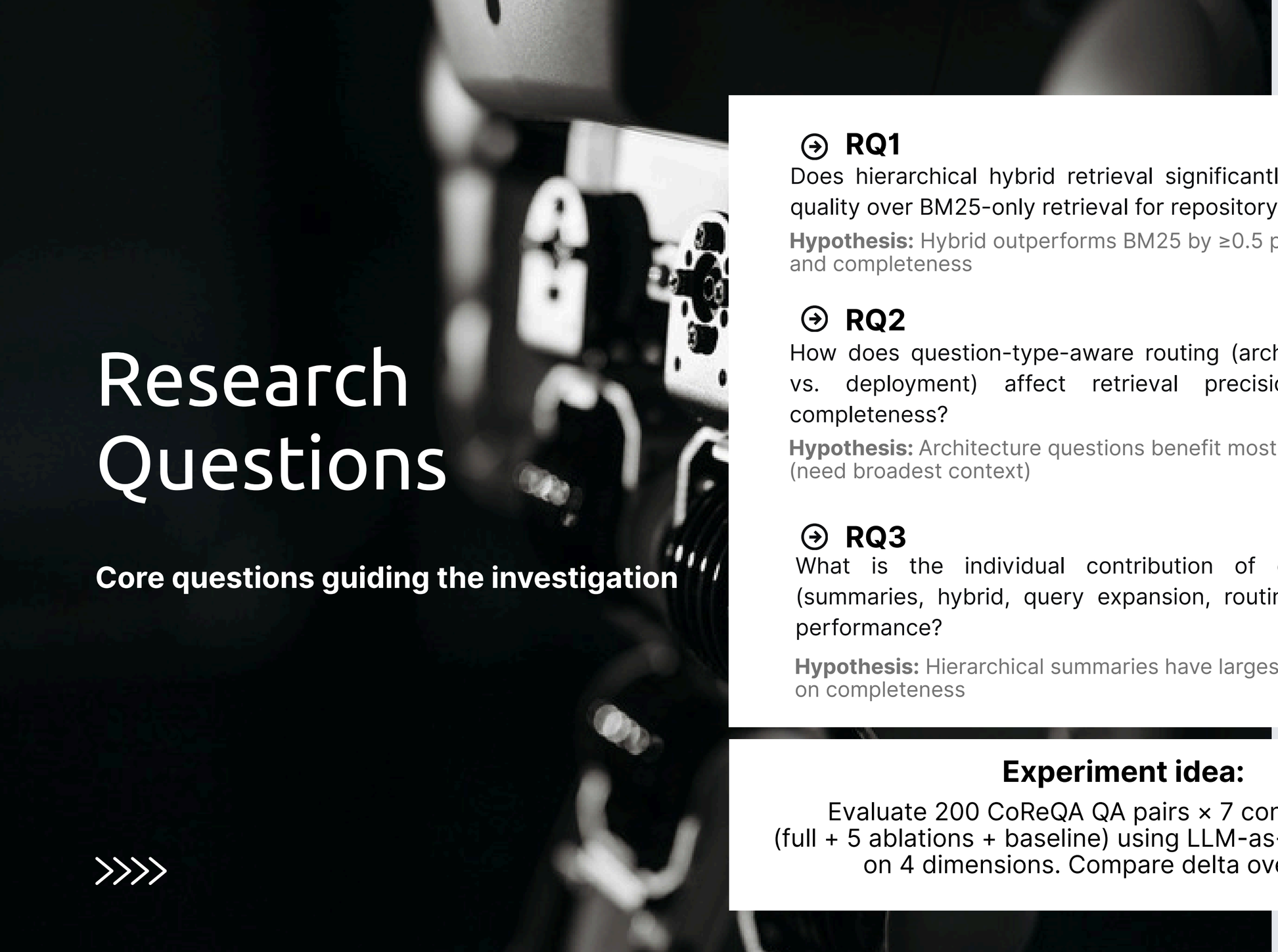
- **Hierarchical Summarization:**
  Generate project, directory, and file level summaries so the system understands the codebase at every level of abstraction

- **Hybrid Retrieval + Question Routing**
  Combine semantic search + BM25 + dependency graph with question-type-aware routing (architecture → broad; logic → focused; deployment → configs)

- **Structured Explanations + Citation Verification**
  Top-down prompts (overview → details) with post-generation verification that every cited file path actually exists

Contribution

(1) Question taxonomy with retrieval strategies per type
(2) Complete training-free RAG pipeline for code explanation
(3) Comprehensive evaluation plan with ablation studies on CoReQA benchmark



Research Questions

Core questions guiding the investigation

RQ1

Does hierarchical hybrid retrieval significantly improve answer quality over BM25-only retrieval for repository-level code QA?

Hypothesis: Hybrid outperforms BM25 by ≥0.5 points on accuracy and completeness

RQ2

How does question-type-aware routing (architecture vs. logic vs. deployment) affect retrieval precision and answer completeness?

Hypothesis: Architecture questions benefit most from routing (need broadest context)

RQ3

What is the individual contribution of each component (summaries, hybrid, query expansion, routing) to overall QA performance?

Hypothesis: Hierarchical summaries have largest individual impact on completeness

Experiment idea:

Evaluate 200 CoReQA QA pairs × 7 configurations (full + 5 ablations + baseline) using LLM-as-a-judge scoring on 4 dimensions. Compare delta over BM25.

Related work and Research Gap

Code QA Benchmarks

- CodeQA (Liu & Wan, 2021): snippet-level
- CoReQA (Chen et al., 2025): 1,563 repo QA
- SWE-QA (Peng et al., 2025): architecture
- CodeRepoQA (2024): multi-turn QA

Gap: Define problem but don't solve retrieval

RAG for Code

- RepoCoder (Zhang et al., 2023): iterative
- DraCo (Cheng et al., 2024): dataflow
- CAST (2025): AST-aware chunking
- CodeRAG-Bench (Wang et al., 2025)

Gap: Optimize for completion, not explanation

Repo Understanding

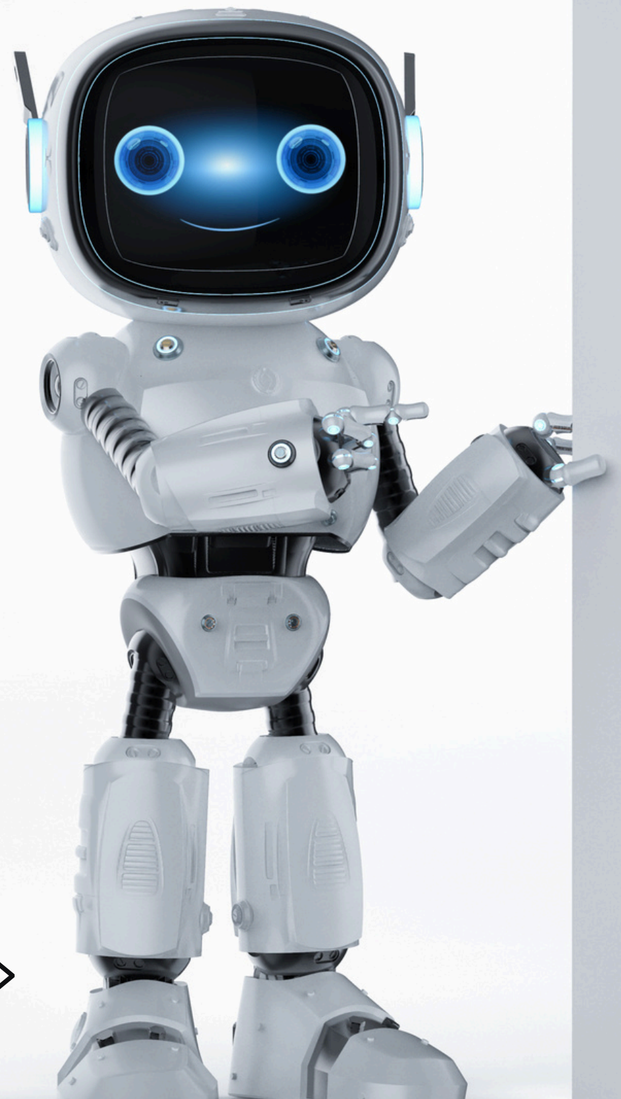
- Hierarchical Summ. (Tufek et al., 2025)
- Strich et al. (ACL 2024): RAG for QA
- CodePlan (Bairi et al., 2024): planning

Gap: Not combined for developer Q&A

CONTRIBUTION:
Bridging all three areas

- combining hierarchical code understanding with hybrid retrieval and question
- type routing in a unified Q&A pipeline,
- then evaluating rigorously on CoReQA's benchmark.

This specific combination has not been explored in prior work.



System Architecture

Four-stage pipeline for repository Q&A

Stage 1: Ingestion & Summarization (Offline Indexing)

GitHub repo → Clone & filter → Tree-sitter AST parse → Hierarchical summaries (project/dir/file) → Embed → ChromaDB + BM25 index + Dep. graph

Stage 2: Hybrid Retrieval + Routing

Developer question → Question classifier (arch/logic/deploy) → Strategy router → Semantic + BM25 + Structural search → RRF fusion → Re-rank

Stage 3: QA Generation

Structured prompt (system instr. → project summary → dir summaries → code chunks → question) → Qwen-7B → Citation verifier → Grounded answer

Stage 4: Conversational Interface

Streamlit chat UI → Conversation memory → Clickable source references (file:line) → Follow-up question support

KEY DESIGN:

Training-free (no fine-tuning needed).
All models used with pre-trained weights.
Innovation is in retrieval architecture, not model training.

Dataset & Data Preprocessing

Models (Free, Run locally)

Primary - CoReQA: 1,563 QA pairs, 176 repos, 4 languages, from real GitHub issues (Chen et al., 2025)
Secondary - SWE-QA: Architectural questions targeting cross-module understanding (Peng et al., 2025)
Development: 1-3 open-source repos (5K–50K lines) for building and tuning the pipeline (Time constraint)

Preprocessing Pipeline

1. **Clone & Filter:** GitPython clone → exclude node_modules, build/, .git, binaries
2. **Tree-sitter Parse:** AST-aware chunking: extract functions, classes, modules (256–512 tokens per chunk)
3. **Non-Code Index:** README, Dockerfile, CI/CD configs, package.json, .env templates
4. **Hierarchical Summaries:** LLM generates project-level → directory-level → file-level summaries (cached)
5. **Embed & Index:** all-mpnet-base-v2 embeddings → ChromaDB + BM25 index + import dependency graph

Models & Training Approach

Models (Chosen free only, Run locally)

Component	Model	Size	Role
Embeddings	all-mpnet-base-v2	110M params	Convert code chunks to vectors for semantic search
Generator	Qwen2.5-7B-Instruct	7B params	Answer questions, generate summaries, classify
Evaluation Judge	Gemini 1.5 Flash (free API)	—	Score answers on 4 dimensions (independent from
Code Parser	Tree-sitter	—	Parse source files into AST for semantic chunking
Keyword Search	rank-bm25	—	BM25 index for exact identifier matching

Training Approach: Training-Free

No fine-tuning required. All models are used with pre-trained weights. This is an intentional design choice:

- Ensures reproducibility - anyone can replicate our results
- Eliminates need for GPU-intensive training (zero budget constraint)
- Demonstrates that retrieval quality is the bottleneck, not model quality
- The only pre-computation is hierarchical summarization (one-time, cached per repo)

Evaluation Metrics

LLM-as-a-Judge Framework (from CoReQA)

Metric	What It Measures	Scale
Accuracy	Factual correctness vs. reference answer	1–10
Completeness	All key aspects of the question covered	1–10
Relevance	Core concern of the question addressed	1–10
Clarity	Logical structure and readability	1–10
Retrieval Recall@k	Fraction of relevant chunks in top-k	0–1.0
Citation Accuracy	% of cited file paths that exist in repo	0–100% (measures hallucination)

How LLM-as-a-Judge works:

1. System generates an answer to a CoReQA question
2. A separate LLM (Gemini Flash) receives: original question + reference answer + system's answer
3. The judge scores system's answer on each dimension (1–10) using chain-of-thought
4. Each evaluation runs 3× and report mean ± std (~5.5% variance per CoReQA)
5. Judge LLM ≠ Generator LLM → avoids self-evaluation bias

Experiment Design & Expected Results

Ablation Study (RQ3)

Configuration	Summaries	Hybrid Retrieval	Query Expansion	Routing
Full RepoQA	✓	✓	✓	✓
• **Summaries**	✗	✓	✓	✓
• **Hybrid (semantic only)**	✓	✗	✓	✓
• **Query expansion**	✓	✓	✗	✓
• **Question Type routing**	✓	✓	✓	✗
BM25 baseline	✗	✗	✗	✗
No retrieval	✗	✗	✗	✗

Expected Results

System	Accuracy	Completeness	Relevance	Clarity
No retrieval (reported*)	6.85	6.27	7.81	8.4
BM25 (reported*)	6.91	6.34	7.86	8.42
RepoQA (our target)	**7.3–7.6**	**6.8–7.1**	**8.0–8.2**	**8.4–8.6**

**Reported by Chen et al. (2025) using GPT-4o*

Key Insight

We measure the **relative improvement** over BM25, not absolute scores.

Using Qwen-7B (free) instead of GPT-4o (paid), our absolute scores may be lower.

But the delta matters:

BM25 compared to GPT-4o: +0.06 acc

BM25 compared to RepoQA: +0.5 acc

This proves retrieval quality matters more than model size.

Discussion

How data and approach answer the research questions

RQ1: Hybrid vs BM25
200 CoReQA pairs evaluated with BM25-only vs full hybird.
4-dimensional scoring reveals which dimensions improve most.

Expected: Completeness gains (multi-file context) + accuracy gains (correct definitions vs. similar-looking text).

RQ2: Question Routing
Manually categorize 200 questions into arch/logic/deploy. Compare routed vs uniform retrieval per category.

Expected: Architecture questions show largest gain (need 5-10 files from multiple directories that hierarchial top-down uniquely provides)

RQ3: Component Ablation
7 configurations (Table), each removing one component. Score drops isolate each contribution.

Expected Ordering: summaries > hybrid > query expansion > routing. Summaries uniquely solve the discovery problem (which directories are relevant).

Practical Impact: If successful, integrates into IDE chat interfaces and developer onboarding tools. Hierarchical summaries double as auto-generated documentation

Summary of Progress

COMPLETED	IN PROGRESS
Literature review	Tree-sitter ingestion pipeline
System architecture design: Tech stack: Python + Tree-sitter + Chroma DB	Hierarchical summarization prompts
CoReQA benchmark analyzed	ChromaDB + BM25 dual index
Question taxonomy defined	Hybrid retrieval + RRF Fusion
Evaluation framework + ablation design	Question classifier + routing
GitHub repo initialized with README	Citation verifier
Completed RAG chatbot (previous assignment)	Streamlit UI + GitHub integration
	CoReQA evaluation (200 subset)
	Ablation study + error analysis
	Final report

Obstacles and Mitigation Strategies

RISK	MITIGATION
Zero budget for API calls	Qwen-7B (local, free) for generation; Gemini Flash (free tier) for evaluation judge
CoReQA dataset access	If unavailable, construct own QA set following their published methodology (GitHub issues + positive comments)
Hierarchical summary quality	Validate manually on 1–2 repos before scaling; iterative prompt refinement
LLM-as-a-judge variance	Run each evaluation 3× and report mean ± std; validate with small human study (if time permits)
Large repository performance	Start with repos <50K lines; incremental indexing; cap directory depth at 3 levels initially
Scope creep	MVP = single-repo Q&A. Multi-repo, architecture diagrams, and interactive navigation are future work

THANK YOU !